

GET TECHY WITH LUCKY
TECH PULSE INSIDER
WEB DEVELOPMENT MASTERCLASS

COMPLETE LEARNING GUIDE

CSS Foundations

Turning HTML Structure Into a Professional, Responsive Interface

WEEK 02 - CSS & RESPONSIVE DESIGN

Selectors, the cascade, the box model, Flexbox, CSS Grid and mobile-first responsive design.

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

Learn • Build • Connect • Grow

Course Information

Field	Detail
Program	Web Development Masterclass
Provider	Get Techy With Lucky / Tech Pulse Insider
Instructor	Lucky Nakola
Week	Week 02 of 08
Module	CSS + Responsive Design
Estimated study time	6-8 hours
Builds on	Week 01 - Web Fundamentals and HTML
Format	Read -> Practise -> Quiz -> Build -> Submit
Assessment	Weekly quiz (auto-scored) and a practical assignment submitted via GitHub

Learning Objectives

By the end of this week you should be able to:

1. Apply CSS selectors correctly and predict which rule wins when several compete.
2. Use the box model, padding, margin and box-sizing to control spacing deliberately.
3. Build one-dimensional layouts with Flexbox and two-dimensional layouts with CSS Grid.
4. Design mobile-first pages that adapt with media queries at breakpoints you chose for a reason.
5. Use custom properties to keep colour, spacing and type consistent across a whole site.
6. Meet basic accessibility requirements for contrast, focus states and form labelling.
7. Rebuild your Week 1 website as a professional, responsive business site.

Prerequisites

Before starting this week, make sure you can already do the following:

- Write valid, semantic HTML using headings, lists, links, images, forms and sectioning elements.
- Explain what the browser does with an HTML file when a page loads.
- Open and use your browser's developer tools to inspect an element.
- Save, organise and open files in a project folder using a code editor.

Table of Contents

This guide is written to be read in order. Each chapter builds on the one before it.

1. Introduction - From Structure to Interface
2. How CSS Works: Syntax, Selectors and the Cascade
3. Specificity and Inheritance - Why Your Style Is Being Ignored
4. The Box Model - Every Element Is a Box
5. Layout with Flexbox - Arranging Things in One Direction
6. Layout with CSS Grid - Arranging Things in Two Directions
7. Responsive Design and Mobile-First Thinking
8. Typography, Colour and Interaction Polish
9. Important Terminology
10. Common Mistakes and How to Avoid Them
11. Accessibility and Professional Considerations
12. Practical Exercise - Rebuild Your Week 1 Site
13. Knowledge Check - Weekly Quiz
14. Assignment - The Responsive Business Site
15. Summary

TECH PULSE INSIDER

Chapter 1 - Introduction - From Structure to Interface

In Week 1 you learned to describe what content means. A heading is a heading, a list is a list, a form is a form. That is HTML's entire job, and it does it well. What HTML cannot do is decide how any of that should look, and browsers fill the gap with a plain default stylesheet: black Times New Roman on white, everything stacked in a single column.

CSS is the language that takes over from there. It is a presentation language: you write rules that describe how elements should be drawn, and the browser applies them. The separation matters. Because structure and presentation live in different files, one stylesheet can restyle a thousand pages, and a screen reader can ignore your styling entirely and still make sense of the page.

WHY THIS MATTERS

Most beginners learn CSS as a list of properties to memorise, then get stuck the first time a style 'does not work'. The properties are the easy part - there is a reference for those. What you actually need is a model of how the browser decides which rule wins and how boxes size themselves. That model is what this guide is built around.

How to Use This Guide

Read this guide with an editor open beside it. Type every example yourself rather than copying and pasting. Typing is slower, and that is the point: it is what turns a concept you have read into a skill you own. When something does not work, resist the urge to look at the answer straight away. Read the error, form a guess about the cause, then test that guess. Debugging is the skill that separates people who can build from people who can only follow.

The Three Ways to Apply CSS

CSS can reach an element by three routes, and only one of them belongs in real work.

```
<!-- 1. Inline - styles on the element itself. Avoid. -->
<p style="color: navy;">Hello</p>

<!-- 2. Internal - a style block in the head. Fine for a one-off demo. -->
<style>
  p { color: navy; }
</style>

<!-- 3. External - a linked file. This is what you use. -->
<link rel="stylesheet" href="styles.css">
```

Three ways to apply CSS. Use the third one.

External stylesheets win because they are cached by the browser across pages, they keep presentation out of your markup, and they can be changed without touching the HTML. Inline styles also carry very high specificity, which makes them painful to override later - a problem you will meet in Chapter 3.

Chapter 2 - How CSS Works: Syntax, Selectors and the Cascade

Anatomy of a Rule

```
selector {  
  property: value; /* one declaration */  
  property: value;  
}  
  
.card {  
  background-color: #ffffff;  
  border-radius: 12px;  
  padding: 24px;  
}
```

A selector, then a block of declarations. Every declaration ends in a semicolon.

The Selectors You Will Actually Use

Selector	Targets	Example	When to reach for it
tag	Every element of that type	p { }	Site-wide defaults such as base type
class	Every element carrying that class	.card { }	Your everyday workhorse
#id	The one element with that id	#main-nav { }	Rarely - prefer classes
A B	B anywhere inside A	nav a { }	Scoping styles to a region
A > B	B that is a direct child of A	ul > li { }	When nesting depth matters
A, B	Both A and B	h1, h2 { }	Sharing one rule across selectors
:hover	An element the pointer is over	a:hover { }	Interactive feedback
:focus-visible	An element focused via keyboard	a:focus-visible { }	Keyboard accessibility
::before	A generated box before the content	li::before { }	Decorative markers

The Cascade

When two rules set the same property on the same element, the browser needs a tie-breaker. It works through three questions in strict order: is one declaration marked !important, is one selector more specific, and finally, which one appears later in the stylesheet.

BEST PRACTICE

Because source order is the last tie-breaker, load your own stylesheet after any third-party one. And treat !important as a signal that something has gone wrong in your architecture, not as a tool. If you need it, the real fix is almost always a cleaner selector.

Chapter 3 - Specificity and Inheritance - Why Your Style Is Being Ignored

Specificity is a score. Count the ID selectors, then the class, attribute and pseudo-class selectors, then the element and pseudo-element selectors. Compare the counts left to right: any number of classes will never beat a single ID.

Selector	IDs	Classes	Elements	Wins against
p	0	0	1	Nothing much
.intro	0	1	0	Any plain tag selector
p.intro	0	1	1	.intro on its own
#lead	1	0	0	Any number of classes
style="..."	-	-	-	Everything except !important

COMMON MISTAKE

The most common cause of 'my CSS is not working' is not a typo - it is a more specific rule elsewhere quietly winning. Open DevTools, select the element, and look at the Styles panel: the losing rule is shown with a line through it. Read the cascade instead of guessing.

Inheritance

Some properties pass down to children automatically. Text-related ones mostly do - color, font-family, font-size, line-height. Box-related ones mostly do not - border, padding, margin, background. This is why setting the font once on the body styles the whole page, while setting a border on the body does not put a border around every paragraph.

```
body {  
  font-family: system-ui, Arial, sans-serif; /* inherited by everything */  
  color: #1a1f2b; /* inherited too */  
  border: 1px solid red; /* NOT inherited */  
}
```

Text properties cascade down; box properties stay put.

Chapter 4 - The Box Model - Every Element Is a Box

Every element the browser draws is a rectangle built from four nested layers: the content, the padding around it, the border around that, and the margin holding neighbours away. Almost every layout bug a beginner hits comes from misreading these four layers.

Layer	Position	Affected by background?	Typical use
Content	Innermost	Yes	The text or image itself
Padding	Inside the border	Yes	Breathing room inside a card or button
Border	Around the padding	It is the border	A visible edge or divider
Margin	Outside the border	No - always transparent	Space between separate elements

The border-box Fix

By default, width sets the width of the content only. Add 24px of padding either side and a 1px border, and a box you declared as 300px actually occupies 350px. That default is a historical accident, and essentially every professional stylesheet turns it off on line one.

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

/* Now this box is exactly 300px wide, padding and border included. */
.card {
  width: 300px;
  padding: 24px;
  border: 1px solid #d6dce5;
}
```

Put this at the top of every stylesheet you write.

Margin Collapse

Vertical margins between siblings do not add up - they collapse to whichever is larger. A 24px bottom margin above a 16px top margin produces 24px of gap, not 40px. This surprises everyone once. The modern way to avoid arguing with it is to use gap on a flex or grid container instead of margins on the children.

Chapter 5 - Layout with Flexbox - Arranging Things in One Direction

Flexbox lays children out along a single axis and distributes the leftover space between them. You make a container flex, and its direct children become flex items. Anything that is essentially a row or essentially a column is a Flexbox job.

```
.site-header {  
  display: flex;  
  justify-content: space-between; /* along the main axis */  
  align-items: center;           /* across the cross axis */  
  gap: 1.5rem;  
  padding: 1rem 1.5rem;  
}  
  
.site-nav {  
  display: flex;  
  gap: 1.25rem;  
  flex-wrap: wrap; /* links drop to a second line instead of overflowing */  
}
```

A standard site header: logo pushed left, navigation pushed right, vertically centred.

Property	Goes on	What it does	Common values
display: flex	Container	Turns on flex layout	flex, inline-flex
flex-direction	Container	Sets the main axis	row, column
justify-content	Container	Distributes space along the main axis	flex-start, center, space-between
align-items	Container	Aligns items across the cross axis	stretch, center, flex-start
flex-wrap	Container	Allows a second line	nowrap, wrap
gap	Container	Space between items	1rem, 12px 24px
flex-grow	Item	Share of extra space to absorb	0, 1
flex-shrink	Item	Willingness to shrink	0, 1
flex-basis	Item	Starting size before growing or shrinking	auto, 220px

NOTE

flex: 1 is shorthand for flex-grow: 1; flex-shrink: 1; flex-basis: 0%. It is the fastest way to say 'let this item soak up whatever space is left'.

Chapter 6 - Layout with CSS Grid - Arranging Things in Two Directions

Grid is the tool for laying out rows and columns at the same time. Where Flexbox lets content decide sizes, Grid lets you define the structure up front and place content into it. Page layouts and card galleries are Grid jobs.

```
.card-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));  
  gap: 1.5rem;  
}
```

A responsive card grid in three lines - and not a single media query.

That rule deserves unpacking, because it is one of the highest-value patterns in modern CSS. `repeat(auto-fit, ...)` tells the browser to fit as many columns as it can. `minmax(260px, 1fr)` says each column must be at least 260px but may grow to share the free space equally. On a phone that yields one column, on a tablet two, on a laptop three or four - decided by the browser from the actual available width rather than by breakpoints you guessed.

WHY THIS MATTERS

This is the difference between reacting to devices and describing intent. You are not saying 'three columns on a laptop'. You are saying 'cards should never be narrower than 260px'. The browser works out the rest, including for screen sizes that did not exist when you wrote it.

Explicit Page Layout

```
.page {  
  display: grid;  
  grid-template-columns: 1fr;  
  grid-template-areas:  
    "header"  
    "main"  
    "sidebar"  
    "footer";  
}  
  
@media (min-width: 900px) {  
  .page {  
    grid-template-columns: 2fr 1fr;  
    grid-template-areas:  
      "header header"  
      "main sidebar"  
      "footer footer";  
  }  
}  
  
.page > header { grid-area: header; }  
.page > main { grid-area: main; }  
.page > aside { grid-area: sidebar; }  
.page > footer { grid-area: footer; }
```

Named grid areas let you redraw a whole page layout by editing an ASCII picture of it.

Choosing Between Flexbox and Grid

Question	Answer suggests
Is it one row or one column?	Flexbox
Do rows and columns both need to line up?	Grid
Should content sizes drive the layout?	Flexbox
Should the layout drive content sizes?	Grid
Is it a navigation bar, button row or toolbar?	Flexbox
Is it a page skeleton, gallery or dashboard?	Grid

They are not rivals and you will regularly nest one inside the other: a Grid page layout whose header is a Flexbox row is completely normal.

Chapter 7 - Responsive Design and Mobile-First Thinking

Responsive design means one site that reshapes itself for the screen it is on. In Kenya and across most of Africa the majority of web traffic is on phones, often on a mid-range Android device over a metered connection. A site that only looks right on a laptop is broken for most of the people who will actually visit it.

Step One: The Viewport Meta Tag

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Without this line in your `<head>`, mobile browsers pretend to be 980px wide and shrink the page.

COMMON MISTAKE

Forgetting the viewport tag makes every media query behave as though the phone were a desktop. Your CSS is fine; the page is being lied to about its width. Check this first when a responsive layout works in the DevTools device toolbar but not on a real phone.

Step Two: Write the Small Screen First

```
/* Base styles: the phone layout. No media query needed. */
.layout {
  display: grid;
  gap: 1.5rem;
}

/* Tablet and up: there is now room for two columns. */
@media (min-width: 48rem) {
  .layout { grid-template-columns: repeat(2, 1fr); }
}
```

```
/* Laptop and up. */
@media (min-width: 64rem) {
  .layout { grid-template-columns: repeat(3, 1fr); }
}
```

Mobile-first: start simple, add structure as space becomes available.

Writing mobile-first is not just a style preference. The phone layout is the simplest one, so it makes the best base, and small-screen devices then download the least CSS work to undo. Writing desktop-first means every media query is spent cancelling something.

Choosing Breakpoints

Do not memorise device widths. Devices change every year and there is no such thing as a standard phone. Instead, widen your browser slowly and watch. The moment the layout starts to look wrong - a line of text gets uncomfortably long, cards get too narrow, a nav bar wraps badly - that width is your breakpoint. It is a property of your design, not of anyone's hardware.

Range	Typical device	What usually changes
Up to 640px	Phones	Single column, stacked nav, larger tap targets
640-1024px	Large phones and tablets	Two columns, nav becomes horizontal
1024px and up	Laptops and desktops	Full multi-column layout, sidebars appear

A starting point for orientation only - always confirm against your own design.

Chapter 8 - Typography, Colour and Interaction Polish

Design Tokens with Custom Properties

Hard-coding #003F9E in forty places means forty edits when the brand colour changes, and one of them will be missed. Declare your values once and reference them everywhere.

```
:root {
  --brand: #003f9e;
  --brand-light: #005ce6;
  --accent: #ffcc00;
  --ink: #1a1f2b;
  --muted: #5b6474;
  --surface: #ffffff;

  --space-sm: 0.5rem;
  --space-md: 1rem;
  --space-lg: 2rem;

  --radius: 12px;
}

.button {
  background-color: var(--brand);
}
```

```
color: var(--surface);
padding: var(--space-sm) var(--space-lg);
border-radius: var(--radius);
}
```

A small design system. One place to change, everywhere updates.

Readable Type

- **Line length.** Between 50 and 75 characters is comfortable. Use max-width: 65ch on body text.
- **Line height.** About 1.5 for body copy, tighter for large headings.
- **Scale, not guesswork.** Pick a small set of sizes and reuse them instead of inventing a new one each time.
- **Never below 16px for body text.** Smaller than that and mobile browsers may zoom your form fields unexpectedly.

States and Transitions

```
.button {
  transition: background-color 180ms ease, transform 180ms ease;
}

.button:hover {
  background-color: var(--brand-light);
}

.button:focus-visible {
  outline: 3px solid var(--accent);
  outline-offset: 2px;
}

@media (prefers-reduced-motion: reduce) {
  .button { transition: none; }
}
```

Hover for the mouse, focus-visible for the keyboard, and an opt-out for motion sensitivity.

COMMON MISTAKE

Never write `outline: none` to remove a focus ring without replacing it with something equally visible. Keyboard users navigate by watching that ring move. Removing it makes a site genuinely unusable for them, and it is one of the most common accessibility failures on the web.

Chapter 9 - Important Terminology

These are the terms that carry the concepts. You do not need to memorise the definitions word for word, but you should be able to explain each one to somebody else in your own words.

Term	Definition	In Plain Words	Example / Use Case
------	------------	----------------	--------------------

Term	Definition	In Plain Words	Example / Use Case
CSS	Cascading Style Sheets - the language that describes how HTML elements should be presented.	The paint, layout and spacing layer of a web page.	color: navy; sets text colour.
Selector	The part of a CSS rule that decides which elements the declarations apply to.	The 'who' of a CSS rule.	p { } selects every paragraph.
Declaration	A single property and value pair inside a rule, ending in a semicolon.	One instruction about how something should look.	margin: 0 auto;
Property	The named aspect of presentation being changed.	What you are changing.	font-size, background-color
Value	The setting given to a property.	What you are changing it to.	16px, #003F9E, flex
Rule set	A selector plus the block of declarations that belongs to it.	One complete CSS instruction.	h1 { font-size: 2rem; }
Cascade	The algorithm that decides which rule wins when several rules target the same element.	The tie-breaker system in CSS.	A later rule beats an earlier one of equal weight.
Specificity	A score calculated from a selector that decides its priority in the cascade.	How specific your aim was.	#id beats .class beats tag.
Inheritance	The way some properties pass automatically from a parent element to its children.	Children copy some styles from parents.	color is inherited; border is not.
Box model	The rule that every element is a rectangle made of content, padding, border and margin.	Every element is a box with layers.	Padding is inside the border, margin outside.
Padding	Space between the content of an element and its border.	Inner breathing room.	padding: 16px;
Margin	Space outside an element's border, separating it from its neighbours.	Outer breathing room.	margin-bottom: 24px;
Border	A line drawn around the padding of an element.	The edge of the box.	border: 1px solid #ddd;
box-sizing	A property that controls whether width includes padding and border.	Makes width mean what you expect.	box-sizing: border-box;
Margin collapse	The behaviour where adjacent vertical margins merge into the larger of the two.	Two stacked margins become one.	24px and 16px become 24px, not 40px.
Block element	An element that starts on a new line and fills the available width.	Takes a whole row.	div, p, section
Inline element	An element that flows within a line of text and only takes the width it needs.	Sits inside a line.	span, a, strong
Flexbox	A one-dimensional layout system for distributing space along a	Lays things out in a row or a column.	display: flex;

Term	Definition	In Plain Words	Example / Use Case
	single axis.		
Main axis	The direction a flex container lays its children out in.	The direction items flow.	Row by default, column if you say so.
Cross axis	The axis perpendicular to the main axis in a flex container.	The 'other' direction.	Vertical when the main axis is horizontal.
justify-content	Distributes free space along the main axis of a flex container.	Spacing along the flow direction.	justify-content: space-between;
align-items	Aligns flex or grid children along the cross axis.	Lining items up across the flow.	align-items: center;
flex-wrap	Controls whether flex items may move onto additional lines.	Lets items drop to a new row.	flex-wrap: wrap;
gap	Space between rows and columns of a flex or grid container.	Gutters without margin hacks.	gap: 1.5rem;
CSS Grid	A two-dimensional layout system that positions items in rows and columns at once.	A real grid for page layout.	display: grid;
grid-template-columns	Defines the number and size of the columns in a grid.	Sets up the columns.	grid-template-columns: repeat(3, 1fr);
fr unit	A fractional unit representing a share of the free space in a grid.	A slice of what is left over.	1fr 2fr gives a one-third / two-thirds split.
minmax()	A grid function setting a minimum and maximum track size.	Never smaller than X, never bigger than Y.	minmax(220px, 1fr)
auto-fit	A repeat() keyword that fits as many tracks as will fit on the line.	Automatic responsive columns.	repeat(auto-fit, minmax(220px, 1fr))
Responsive design	Building one site that adapts its layout to any screen size.	One site that works on every device.	A three-column grid becoming one column on a phone.
Media query	A CSS rule that applies styles only when a condition about the device is true.	'Only when the screen is at least this wide.'	@media (min-width: 768px) { }
Breakpoint	The screen width at which a layout is changed.	Where the design switches shape.	768px is a common tablet breakpoint.
Mobile-first	Writing base styles for small screens and adding complexity upward.	Design for the phone first.	Base CSS, then min-width media queries.
Viewport	The visible area of a web page inside the browser window.	The part of the page you can see.	The viewport meta tag controls it on phones.
Viewport meta tag	An HTML tag telling mobile browsers to use the device width rather than pretend to be a desktop.	Turns responsive CSS on for phones.	<meta name="viewport" content="width=device-width, initial-scale=1">
Relative unit	A unit sized against something else rather than a fixed number of pixels.	Scales with context.	rem, em, %, vw, vh
rem	A unit equal to the root element's font size.	A consistent scale for the whole page.	1rem is usually 16px.

Term	Definition	In Plain Words	Example / Use Case
em	A unit equal to the font size of the current element.	Scales with local text size.	padding: 0.5em;
Custom property	A reusable value stored in CSS and read with var().	A CSS variable.	--brand: #003F9E; color: var(--brand);
Pseudo-class	A keyword on a selector describing a special state of an element.	Style based on state.	a:hover, input:focus
Pseudo-element	A keyword that styles a specific part of an element.	Style part of an element.	p::first-line
Transition	A smooth animation between two states of a property over time.	Makes changes fade rather than snap.	transition: background-color 200ms ease;
Flow	The default way the browser positions elements one after another.	The natural order of the page.	Blocks stack, inlines sit side by side.
position: relative	Positions an element relative to where it would normally sit.	Nudge it, keep its space.	Used as an anchor for absolute children.
position: absolute	Removes an element from flow and positions it against its nearest positioned ancestor.	Take it out and place it exactly.	A badge pinned to a card corner.
z-index	Controls which overlapping positioned element is drawn on top.	Stacking order.	z-index: 10;
Stylesheet	A separate .css file linked from HTML holding the site's styles.	Where your CSS lives.	<link rel="stylesheet" href="styles.css">
Contrast ratio	The measured difference in luminance between text and its background.	How readable text is against its background.	4.5:1 is the minimum for body text.

Chapter 10 - Common Mistakes and How to Avoid Them

Mistake	What goes wrong	The fix
Skipping box-sizing: border-box	Boxes are wider than declared and layouts overflow	Set it globally at the top of every stylesheet
Reaching for limportant	Specificity wars that get worse with every fix	Find the rule that is winning and write a cleaner selector
Fixed pixel widths on containers	Horizontal scrollbars on phones	Use max-width with a percentage or fr
Desktop-first media queries	Every query spends its time undoing desktop styles	Write the small screen as the base, then add with min-width
Removing focus outlines	Keyboard users lose all sense of position	Restyle :focus-visible rather than deleting the outline
Grey text on white for body copy	Fails contrast, unreadable in sunlight	Keep body text at 4.5:1 or better

Mistake	What goes wrong	The fix
Deep descendant selectors like .a .b .c span	Fragile, slow to reason about, hard to override	Give the element a class and target it directly
Margins on grid or flex children for gutters	Margin collapse and unwanted edge spacing	Use gap on the container
No viewport meta tag	Media queries never match on real phones	Add it to the head of every page
Styling before the HTML is semantic	Divs everywhere, nothing meaningful to select	Fix the markup first - good HTML makes CSS shorter

Chapter 11 - Accessibility and Professional Considerations

Accessibility is not an optional extra bolted on at the end. It is part of the definition of a working website, it is increasingly a legal requirement, and the same work that helps a screen reader user also helps somebody on a cracked phone in bright sunlight.

- **Contrast.** Body text needs 4.5:1 against its background; large text needs 3:1. Check with your browser's contrast tool, not your eyes.
- **Focus.** Every interactive element must have a clearly visible focus state. Tab through your own page and watch.
- **Colour is never the only signal.** If an error is shown only in red, a colour-blind user sees nothing. Add an icon or text.
- **Respect user settings.** Honour prefers-reduced-motion and prefers-color-scheme rather than overriding them.
- **Tap targets.** Aim for at least 44 by 44 CSS pixels for anything touched on a phone.
- **Do not hide content with display: none if it should still be announced.** Use a visually-hidden utility class instead.

Performance Is Part of the Design

On a metered mobile connection, every kilobyte is somebody's money. Keep stylesheets lean, avoid loading five font weights when two will do, and prefer CSS transitions over JavaScript animation. A beautiful site that takes eight seconds to load on 3G is not a beautiful site.

Chapter 12 - Practical Exercise - Rebuild Your Week 1 Site

Work through these in order. Each step should take 20 to 40 minutes. Do not move on until the current step actually works in your browser.

8. Create styles.css, link it from your Week 1 HTML, and confirm it is loading by setting body { background: #eee; }.
9. Add the global border-box reset and remove default body margin.
10. Declare your palette, spacing scale and radius as custom properties on :root.

11. Set base typography on body: font family, colour, line-height 1.5, and a max-width on your text container.
12. Build the site header with Flexbox: name on the left, navigation on the right, vertically centred.
13. Style the hero: generous padding, a clear heading hierarchy, and one prominent call-to-action button.
14. Convert your services section into a Grid with `repeat(auto-fit, minmax(260px, 1fr))` and a gap.
15. Style the cards: white surface, subtle border, radius, padding, and a hover state with a transition.
16. Lay out the about section as a two-column Grid above 900px and a single column below it.
17. Style the contact form: full-width fields, visible labels, comfortable padding, and a focus-visible ring.
18. Style the footer to match the header.
19. Open DevTools, switch to the device toolbar, and check 360px, 768px and 1280px. Fix every overflow.
20. Tab through the whole page with the keyboard. Confirm you can always see where you are.
21. Run the contrast checker on your body text and your button. Adjust until both pass.

BEST PRACTICE

Commit after each numbered step with a short message describing what you did. By Week 4 you will be doing this with Git properly, and building the habit now costs you nothing.

Chapter 13 - Knowledge Check - Weekly Quiz

Answer without looking back through the guide first. Then check, and re-read any chapter where you got something wrong. The same questions appear in the LMS quiz for this week, where they are scored automatically.

Pass mark: 70% | *Answers are at the end of this chapter.*

Q1. Which CSS declaration makes an element's stated width include its padding and border?

- a) `box-sizing: content-box;`
- b) `box-sizing: border-box;`
- c) `width: auto;`
- d) `display: block;`

Q2. A rule written with `#price` beats a rule written with `.price`. Why?

- a) It appears later in the file
- b) IDs have higher specificity than classes
- c) IDs load faster
- d) Classes only work on text

Q3. In a mobile-first stylesheet, which media query direction do you normally write?

- a) `max-width`, adding styles as screens shrink
- b) `min-width`, adding styles as screens grow
- c) Both at once for every rule

d) Neither - media queries are optional

Q4. Which layout tool is the better fit for a page-level layout with both rows and columns?

- a) Flexbox
- b) CSS Grid
- c) Floats
- d) Tables

Q5. Margin is the space inside an element, between its content and its border.

True / False

Q6. Without the viewport meta tag, a mobile browser will still apply your media queries correctly.

True / False

Q7. The value 1fr in a grid means one pixel of free space.

True / False

Q8. Explain, in your own words, the difference between Flexbox and CSS Grid, and give one situation where each is the better choice.

Short answer - write two or three sentences.

Q9. Your paragraph text is not turning blue even though you wrote `p { color: blue; }`. List three things you would check, in order.

Short answer - write two or three sentences.

Q10. Why is a contrast ratio of at least 4.5:1 required for body text?

Short answer - write two or three sentences.

Answer Key

Mark your own work honestly - the goal is to find your gaps, not to score well.

#	Answer	Why
Q1	b - box-sizing: border-box;	With border-box, padding and border are drawn inside the width you set, so a 300px box stays 300px wide.
Q2	b - IDs have higher specificity than classes	Specificity is scored before source order. An ID outranks any number of classes, which is why over-using IDs makes CSS hard to maintain.
Q3	b - min-width, adding styles as screens grow	Mobile-first means the base CSS is the phone layout, and min-width queries layer on the extra structure larger screens can afford.
Q4	b - CSS Grid	Grid is two-dimensional. Flexbox is one-dimensional and is better for a single row or column, such as a navigation bar.
Q5	False	That describes padding. Margin is the space outside the border, separating the element from its neighbours.
Q6	False	Mobile browsers default to a pretend desktop width and scale the page down, so min-width queries match the wrong width. The viewport tag is what makes responsive CSS work.
Q7	False	fr is a fraction of the remaining free space in the container, not a fixed length. <code>repeat(3, 1fr)</code> makes three equal columns whatever the container

#	Answer	Why
		width.
Q8	Flexbox lays items out along a single axis and lets content decide sizes; Grid defines rows and columns up front. Flexbox suits a navigation bar or a row of buttons; Grid suits a whole page layout or a card gallery.	Being able to justify a layout choice matters more than memorising properties. Both answers must mention one versus two dimensions.
Q9	1) Is the stylesheet actually linked and loading? 2) Is another rule with higher specificity or a later position overriding it? 3) Is the selector matching the element at all, or is the colour set inline on the element?	This is the standard CSS debugging order: does the file load, does the selector match, and does something else win the cascade. Browser DevTools shows all three.
Q10	Because text below that ratio is hard or impossible to read for people with low vision, in bright sunlight, or on poor screens. It is the WCAG AA minimum for normal-sized text.	Accessibility is a professional requirement, not decoration. Low-contrast grey-on-white is one of the most common real-world failures.

Chapter 14 - Assignment - The Responsive Business Site

Objective

Turn the static HTML site you built in Week 1 into a fully responsive, professionally styled website using nothing but hand-written CSS. No frameworks this week - you need to understand the machinery before Week 3 hands you a shortcut for it.

Scenario

A small business in your town - a salon, a hardware shop, a tutoring centre, a bakery - has an HTML page with no styling. Customers mostly find them on a phone. Your job is to make the site look credible and work properly on a 360px phone, a tablet, and a laptop.

Requirements

- A sticky or fixed site header containing the business name and navigation links.
- A hero section with a heading, a short supporting sentence, and one clear call-to-action button.
- A services or products section built as a responsive card grid: one column on phones, two on tablets, three on laptops.
- An about section combining text and at least one image, laid out side by side on wide screens and stacked on narrow ones.
- A contact section with a styled form (name, email, message) and a visible focus state on every field.
- A footer with copyright, contact details, and social links.
- A consistent colour palette and type scale defined once using CSS custom properties.
- Hover and focus states on every interactive element, with a transition rather than a hard snap.

Expected Output

One HTML file and one linked CSS file that render a complete, polished business website. At 360px wide there must be no horizontal scrollbar and no overlapping text. At 1280px the layout must use the extra width rather than stretching a single column across the screen.

Technical Requirements

- Mobile-first CSS: base styles for small screens, then min-width media queries.
- At least two breakpoints, chosen because the layout breaks there - not copied from a tutorial.
- Flexbox for the header and any single-direction rows.
- CSS Grid for the card section and the overall page layout.
- box-sizing: border-box applied globally.
- Colours, spacing and font sizes driven by CSS custom properties declared on :root.
- Relative units (rem, %, fr) for sizing; pixels only where a fixed value is genuinely correct, such as a 1px border.
- All body text at a contrast ratio of 4.5:1 or better against its background.
- No inline style attributes and no !important anywhere in the stylesheet.

Submission Instructions

22. Create a public GitHub repository named week-02-responsive-site.
23. Commit your work in meaningful steps, not as one giant commit at the end.
24. Write a README.md explaining the business, your colour and type choices, and which breakpoints you chose and why.
25. Include two screenshots in the README: the phone layout and the desktop layout.
26. Submit the repository link on the Week 2 assignment page in the student portal.

Evaluation Criteria

Criterion	What Earns Full Marks	Weight
Responsiveness	Layout adapts cleanly at every width from 320px to 1440px with no overflow or overlap	30%
Layout technique	Flexbox and Grid each used where they are the right tool, not forced	20%
Visual quality	Consistent spacing scale, readable type hierarchy, coherent palette	20%
Code quality	Organised, commented, custom properties used, no !important, no inline styles	15%
Accessibility	Contrast passes, focus states visible, form labels present and associated	10%
Documentation	README explains the reasoning behind the design decisions	5%

Bonus Challenges

- Add a dark mode using CSS custom properties and the prefers-color-scheme media query.
- Build a responsive hamburger navigation that works with CSS only (the checkbox technique).
- Add a subtle entry animation on the hero, respecting prefers-reduced-motion.
- Deploy the site to GitHub Pages and put the live link at the top of your README.

Chapter 15 - Summary

- CSS is a presentation language. Structure stays in HTML; how it looks belongs here.
- When a style does not apply, the cause is nearly always specificity or source order - read the cascade in DevTools instead of guessing.
- Every element is a box of content, padding, border and margin. Set box-sizing: border-box globally so width means what you expect.
- Flexbox handles one direction; Grid handles two. Nesting them is normal and expected.
- repeat(auto-fit, minmax(...)) gives you a responsive grid without any media queries.
- Write mobile-first and choose breakpoints from where your design breaks, not from a list of device widths.
- Custom properties turn scattered magic numbers into a small design system you can change in one place.
- Contrast, visible focus states and respect for user preferences are requirements, not polish.

You are not expected to remember everything in this guide. Professional developers look things up constantly. What you are expected to build is a mental model: knowing that a thing exists, roughly how it behaves, and where to look it up when you need the detail.

Further Reading and References

Documentation changes faster than any printed guide. Learning to read the official docs is part of the skill you are building.

- **MDN CSS reference - the authoritative documentation** - <https://developer.mozilla.org/en-US/docs/Web/CSS>
- **MDN Learn: CSS layout** - https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout
- **A Complete Guide to Flexbox - CSS-Tricks** - <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- **A Complete Guide to Grid - CSS-Tricks** - <https://css-tricks.com/snippets/css/complete-guide-grid/>
- **WebAIM contrast checker** - <https://webaim.org/resources/contrastchecker/>
- **Web Content Accessibility Guidelines (WCAG) quick reference** - <https://www.w3.org/WAI/WCAG21/quickref/>
- **Can I Use - browser support tables** - <https://caniuse.com/>